

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Hamka Arifani

NIM. 2410817110006

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
Mei 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect To The Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile . Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Hamka Arifani
NIM : 2410817110006

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
SOAL 1	1
A. Source Code	1
B. Output Program	64
C. Pembahasan	69
TAUTAN GIT.....	80

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Logging Timber Soal 1 Modul 5.....	64
Gambar 2. Tampilan Halaman Home Aplikasi FILAFI Modul 5.....	65
Gambar 3. Tampilan Halaman Detail Aplikasi FILAFI Modul 5.....	66
Gambar 4. Tampilan Halaman Pemilihan Bahasa Aplikasi FILAFI Modul 5.....	67
Gambar 5. Tampilan Halaman Home Berbahasa Inggris Aplikasi FILAFI Modul 5.	68
Gambar 6. Tampilan Halaman Detail Berbahasa Inggris Modul 5.....	69

DAFTAR TABEL

Tabel 1. Source Code Soal 1 MainActivity.kt Modul 5.....	1
Tabel 2. Source Code Soal 1 MyApplication.kt Modul 5.....	3
Tabel 3. Source Code Soal 1 SettingPreferences.kt Modul 5	4
Tabel 4. Source Code Soal 1 FilmEntity.kt Modul 5	5
Tabel 5. Source Code Soal 1 FilmDao.kt Modul 5	6
Tabel 6. Source Code Soal 1 FilmDatabase.kt Modul 5	7
Tabel 7. Source Code Soal 1 ApiService.kt Modul 5	8
Tabel 8. Source Code Soal 1 NetworkModule.kt Modul 5.....	9
Tabel 9. Source Code Soal 1 FilmDto.kt Modul 5.....	12
Tabel 10. Source Code Soal 1 ErrorStatus Modul 5	13
Tabel 11. Source Code Soal 1 FilmRepository.kt Modul 5	13
Tabel 12. Source Code Soal 1 Film.kt Modul 5.....	17
Tabel 13. Source Code Soal 1 Nav_Graph.kt Modul 5.....	18
Tabel 14. Source Code Soal 1 HomeScreen.kt Modul 5.....	20
Tabel 15. Source Code Soal 1 HomeViewModel.kt Modul 5	35
Tabel 16. Source Code Soal 1 HomeViewModelFactory.kt Modul 5	37
Tabel 17. Source Code Soal 1 DetailScreen.kt Modul 5.....	38
Tabel 18. Source Code Soal 1 DetailViewModel.kt Modul 5	50
Tabel 19. Source Code Soal 1 DetailViewModelFactory Modul 5	51
Tabel 20. Source Code Soal 1 LanguageScreen.kt Modul 5.....	53
Tabel 21. Source Code Soal 1 LanguageViewModel.kt Modul 5	57
Tabel 22. Source Code Soal 1 LanguageViewModelFactory Modul 5	59
Tabel 23. Source Code Soal 1 HeaderScreen.kt Modul 5.....	60

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started> \
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code Compose

MainActivity

Tabel 1. Source Code Soal 1 MainActivity.kt Modul 5

1.	<code>package com.example.listcompose</code>
2.	
3.	<code>import android.os.Bundle</code>
4.	<code>import androidx.activity.compose.setContent</code>
5.	<code>import androidx.appcompat.app.AppCompatActivity</code>

6.	import androidx.compose.material3.MaterialTheme
7.	import androidx.compose.material3.Surface
8.	import androidx.navigation.compose.rememberNavController
9.	import com.example.listcompose.ui.navigation.Nav_Graph
10.	import com.example.listcompose.ui.theme.ListComposeTheme
11.	
12.	class MainActivity : AppCompatActivity() {
13.	override fun onCreate(savedInstanceState: Bundle?) {
14.	super.onCreate(savedInstanceState)
15.	setContent {
16.	ListComposeTheme {
17.	val navController =
18.	rememberNavController()
19.	Surface(
20.	color =
21.	MaterialTheme.colorScheme.background
22.	) {
23.	Nav_Graph(
24.	navController = navController,
25.	startDestination = "home"
26.	)
	}
	}

27.	}
28.	}
29.	}

MyApplication

Tabel 2. Source Code Soal 1 MyApplication.kt Modul 5

1.	package com.example.listcompose
2.	
3.	import android.app.Application
4.	import android.content.pm.ApplicationInfo
5.	import timber.log.Timber
6.	import com.example.listcompose.R
7.	
8.	class MyApplication : Application() {
9.	companion object {
10.	var isDebug: Boolean = false
11.	var tmdbApiKey : String = ""
12.	}
13.	override fun onCreate() {
14.	super.onCreate()
15.	isDebug = (applicationInfo.flags and ApplicationInfo.FLAG_DEBUGGABLE) != 0
16.	if (isDebug){
17.	Timber.plant(Timber.DebugTree())

18.	}
19.	
20.	tmdbApiKey = getString(R.string.TMDB_API_KEY)
21.	}
22.	}

SettingPreferences.kt

Tabel 3. Source Code Soal 1 SettingPreferences.kt Modul 5

1.	package com.example.listcompose.data.local.pref
2.	
3.	import android.content.Context
4.	import androidx.datastore.core.DataStore
5.	import androidx.datastore.preferences.core.Preferences
6.	import androidx.datastore.preferences.core.booleanPreferencesKey
7.	import androidx.datastore.preferences.core.edit
8.	import androidx.datastore.preferences.core.stringPreferencesKey
9.	import androidx.datastore.preferences.preferencesDataStore
10.	import kotlinx.coroutines.flow.Flow
11.	import kotlinx.coroutines.flow.map
12.	
13.	val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "filafi_seetting")

14.	class SettingPreferences(private val context: Context){
15.	private val LANGUAGE_KEY =
	stringPreferencesKey("app_language")
16.	
17.	val getLanguageSetting: Flow<String> =
	context.dataStore.data
18.	.map { preferences -> preferences[LANGUAGE_KEY]?:
	"en-US" }
19.	suspend fun saveLanguageSetting(languageCode: String){
20.	context.dataStore.edit { preferences ->
	preferences[LANGUAGE_KEY]= languageCode }
21.	}
22.	}

FilmEntity

Tabel 4. Source Code Soal 1 FilmEntity.kt Modul 5

1.	package com.example.listcompose.data.local.room
2.	
3.	import androidx.room.ColumnInfo
4.	import androidx.room.Entity
5.	import androidx.room.PrimaryKey
6.	
7.	@Entity(tableName = "movies")
8.	data class FilmEntity (

9.	@PrimaryKey val id: Int,
10.	@ColumnInfo(name="title") val title: String,
11.	@ColumnInfo(name="poster_path") val posterPath: String?,
12.	@ColumnInfo(name="release_date") val releaseDate: String?
13.	)

FilmDao

Tabel 5. Source Code Soal 1 FilmDao.kt Modul 5

1.	package com.example.listcompose.data.local.room
2.	
3.	import androidx.room.Dao
4.	import androidx.room.Entity
5.	import androidx.room.Insert
6.	import androidx.room.OnConflictStrategy
7.	import androidx.room.Query
8.	import kotlinx.coroutines.flow.Flow
9.	
10.	@Dao
11.	interface FilmDao {
12.	@Query("SELECT * FROM movies")
13.	fun getAllMovies(): Flow<List<FilmEntity>>
14.	@Query("SELECT * FROM movies WHERE id = :movieId")

15.	fun getMovieById(movieId: Int): Flow<FilmEntity>
16.	@Insert(onConflict = OnConflictStrategy.REPLACE)
17.	suspend fun insertMovies(movies: List<FilmEntity>): List<Long>
18.	@Query("DELETE FROM movies")
19.	suspend fun clearAllMovies(): Int
20.	}

FilmDatabase

Tabel 6. Source Code Soal 1 FilmDatabase.kt Modul 5

1.	package com.example.listcompose.data.local.room
2.	
3.	import android.content.Context
4.	import androidx.room.Database
5.	import androidx.room.Room
6.	import androidx.room.RoomDatabase
7.	
8.	@Database(9. entities = [FilmEntity::class], 10. version = 1, 11. exportSchema = false 12.)
13.	abstract class FilmDatabase: RoomDatabase(){
14.	abstract fun filmDao(): FilmDao

15.	
16.	companion object{
17.	@Volatile
18.	private var INSTANCE: FilmDatabase? = null
19.	
20.	fun getDatabase(context: Context): FilmDatabase {
21.	return INSTANCE ?: synchronized(this){
22.	val instance = Room.databaseBuilder(
23.	context.applicationContext,
24.	FilmDatabase::class.java,
25.	"filafi_db"
26.	).build()
27.	INSTANCE = instance
28.	instance
29.	}
30.	}
31.	}
32.	}

Tabel 7. Source Code Soal 1 ApiService.kt Modul 5

1.	package com.example.listcompose.data.remote.api
2.	import com.example.listcompose.data.remote.dto.FilmDto
3.	import retrofit2.Response

4.	import retrofit2.http.GET
5.	import retrofit2.http.Path
6.	import retrofit2.http.Query
7.	interface ApiService {
8.	@GET("movie/{movie_id}")
9.	suspend fun getMovieDetail (
10.	@Path("movie_id") movieId : Int,
11.	@Query("language") language: String
12.	): Response<FilmDto>
13.	}

NetworkModule

Tabel 8. Source Code Soal 1 NetworkModule.kt Modul 5

1.	package com.example.listcompose.data.remote.api
2.	
3.	import com.example.listcompose.MyApplication
4.	import com.jakewharton.retrofit2.converter.kotlinx.serialization. asConverterFactory
5.	import kotlinx.serialization.json.Json
6.	import okhttp3.Interceptor
7.	import okhttp3.MediaType.Companion.toMediaType
8.	import okhttp3.OkHttpClient
9.	import okhttp3.logging.HttpLoggingInterceptor

```
10. import retrofit2.Retrofit
11. import java.util.concurrent.TimeUnit
12.
13. object NetworkModule {
14.     private const val BASE_URL =
15.         "https://api.themoviedb.org/3/"
16.
17.     private val loggingInterceptor =
18.         HttpLoggingInterceptor().apply {
19.             level = if (MyApplication.isDebug) {
20.                 HttpLoggingInterceptor.Level.BODY
21.             } else {
22.                 HttpLoggingInterceptor.Level.NONE
23.             }
24.         }
25.
26.     private val authInterceptor= Interceptor{chain ->
27.         val originalRequest = chain.request()
28.         val authorizedUrl =
29.             originalRequest.url.newBuilder()
30.                 .addQueryParameter("api_key",
31.                     MyApplication.tmdbApiKey)
32.                 .build()
33.         val newRequest = originalRequest.newBuilder()
```

30.	<code>.url(authorizedUrl)</code>
31.	<code>.build()</code>
32.	<code>chain.proceed(newRequest)</code>
33.	<code>}</code>
34.	
35.	<code>private val okHttpClient = OkHttpClient.Builder()</code>
36.	<code>.addInterceptor (authInterceptor)</code>
37.	<code>.addInterceptor(loggingInterceptor)</code>
38.	<code>.connectTimeout(30, TimeUnit.SECONDS)</code>
39.	<code>.readTimeout(30, TimeUnit.SECONDS)</code>
40.	<code>.writeTimeout(30, TimeUnit.SECONDS)</code>
41.	<code>.build()</code>
42.	
43.	<code>private val json = Json {</code>
44.	<code>ignoreUnknownKeys = true</code>
45.	<code>coerceInputValues = true</code>
46.	<code>isLenient = true</code>
47.	<code>}</code>
48.	
49.	<code>private val retrofit= Retrofit.Builder()</code>
50.	<code>.baseUrl(BASE_URL)</code>
51.	<code>.client(okHttpClient)</code>

52.	<code>.addConverterFactory(json.asConverterFactory("application/ json".toMediaType()))</code>
53.	<code>.build()</code>
54.	
55.	<code>val tmdbService : ApiService = retrofit.create(ApiService::class.java)</code>
56.	
57.	<code>}</code>

FilmDto

Tabel 9. Source Code Soal 1 FilmDto.kt Modul 5

1.	<code>package com.example.listcompose.data.remote.dto</code>
2.	
3.	<code>import kotlinx.serialization.SerialName</code>
4.	<code>import kotlinx.serialization.Serializable</code>
5.	
6.	<code>@Serializable</code>
7.	<code>data class FilmDto (</code>
8.	<code> @SerialName("id")</code>
9.	<code> val id : Int,</code>
10.	<code> @SerialName("title")</code>
11.	<code> val title : String,</code>
12.	<code> @SerialName("poster_path")</code>

13.	val posterPath : String?,
14.	@SerializedName("release_date")
15.	val releaseDate : String?
16.	)

ErrorStatus

Tabel 10. Source Code Soal 1 ErrorStatus Modul 5

1.	package com.example.listcompose.data.remote.response
2.	
3.	enum class ErrorStatus {
4.	NOT_FOUND,
5.	NO_INTERNET,
6.	UNAUTHORIZED,
7.	SERVER_ERROR,
8.	UNKNOWN_ERROR
9.	}

FilmRepository

Tabel 11. Source Code Soal 1 FilmRepository.kt Modul 5

1.	package com.example.listcompose.data.repository
2.	
3.	import coil.network.HttpException
4.	import
	com.example.listcompose.data.local.pref.SettingPreferences

5.	import com.example.listcompose.data.local.room.FilmDao
6.	import com.example.listcompose.data.local.room.FilmEntity
7.	import com.example.listcompose.data.remote.api.ApiService
8.	import com.example.listcompose.data.remote.response.ErrorStatus
9.	import com.example.listcompose.domain.model.Film
10.	import com.example.listcompose.domain.model.toDomain
11.	import kotlinx.coroutines.flow.Flow
12.	import kotlinx.coroutines.flow.first
13.	import kotlinx.coroutines.flow.map
14.	import okio.IOException
15.	import timber.log.Timber
16.	
17.	class FilmRepository(
18.	private val apiService: ApiService,
19.	private val filmDao: FilmDao,
20.	private val settingPreferences: SettingPreferences
21.	) {
22.	fun getAllMovies(): Flow<List<Film>>{
23.	return filmDao.getAllMovies().map { entityList->
24.	entityList.map { entity -> entity.toDomain() }
25.	}
26.	}

```

27.
28.     fun getMovieById(movieId: Int): Flow<Film> {
29.         return filmDao.getMovieById(movieId).map { entity
->
30.             entity.toDomain()
31.         }
32.     }
33.
34.     suspend fun refreshMovies(): ErrorStatus? {
35.         return try{
36.             val activeLanguage =
settingPreferences.getLanguageSetting.first()
37.             val filmIds = listOf(274, 1393326, 687163,
1310741, 812583)
38.             val filmEntities = mutableListOf<FilmEntity>()
39.
40.             for (id in filmIds) {
41.                 val response =
apiService.getMovieDetail(movieId = id, language =
activeLanguage)
42.
43.                 if (response.isSuccessful) {
44.                     response.body()?.let { responseBody ->
45.                         filmEntities.add(

```

```

46.         FilmEntity(
47.             id = responseBody.id,
48.             title =
responseBody.title,
49.             posterPath =
responseBody.posterPath,
50.             releaseDate =
responseBody.releaseDate
51.         )
52.     )
53. }
54. }
55. }
56.
57.     if(filmEntities.isNotEmpty()){
58.         filmDao.clearAllMovies()
59.         filmDao.insertMovies(filmEntities)
60.         null
61.     }else{
62.         HttpStatus.SERVER_ERROR
63.     }
64. }catch (e: IOException) {
65.     HttpStatus.NO_INTERNET
66. } catch (e: HttpException) {

```

67.	ErrorStatus.SERVER_ERROR
68.	} catch (e: Exception) {
69.	Timber.e(e, "Penyebab Error:")
70.	ErrorStatus.UNKNOWN_ERROR
71.	}
72.	}
73.	}

Film.kt

Tabel 12. Source Code Soal 1 Film.kt Modul 5

1.	package com.example.listcompose.domain.model
2.	
3.	import com.example.listcompose.data.local.room.FilmEntity
4.	
5.	data class Film (
6.	val id: Int,
7.	val title: String,
8.	val posterPath: String?,
9.	val releaseDate: String?
10.	)
11.	
12.	fun FilmEntity.toDomain(): Film{
13.	return Film(

14.	id = id,
15.	title = title,
16.	posterPath = posterPath,
17.	releaseDate = releaseDate
18.	)
19.	}

Nav_Graph

Tabel 13. Source Code Soal 1 Nav_Graph.kt Modul 5

1.	package com.example.listcompose.ui.navigation
2.	
3.	import androidx.compose.runtime.Composable
4.	import androidx.compose.ui.res.stringResource
5.	import androidx.lifecycle.viewmodel.compose.viewModel
6.	import androidx.navigation.NavHostController
7.	import androidx.navigation.NavType
8.	import androidx.navigation.compose.NavHost
9.	import androidx.navigation.compose.composable
10.	import androidx.navigation.navArgument
11.	import com.example.listcompose.ui.screen.detail.DetailScreen
12.	import com.example.listcompose.ui.screen.home.HomeScreen
13.	import com.example.listcompose.ui.screen.language.LanguageScreen

```

14. import
    com.example.listcompose.ui.screen.home.HomeViewModel
15. import
    com.example.listcompose.ui.screen.home.HomeViewModelFactor
    y
16. import com.example.listcompose.R
17. import
    com.example.listcompose.ui.screen.detail.DetailViewModel
18. import
    com.example.listcompose.ui.screen.detail.DetailViewModelFa
    ctory
19.
20. @Composable
21. fun Nav_Graph(
22.     navController: NavHostController,
23.     startDestination : String = "home"
24. ) {
25.     NavHost(
26.         navController = navController,
27.         startDestination = startDestination
28.     ) {
29.         composable("home") {
30.             HomeScreen(navController = navController)
31.         }
32.

```


33.	composable(
34.	route = "detail/{filmId}",
35.	arguments = listOf(navArgument("filmId") {
	type = NavType.IntType })
36.	) { backStackEntry ->
37.	val filmId =
	backStackEntry.arguments?.getInt("filmId") ?: 0
38.	
39.	DetailScreen(
40.	filmId = filmId,
41.	navController = navController
42.	)
43.	}
44.	composable("language") {
45.	LanguageScreen(navController = navController)
46.	}
47.	}
48.	}

HomeScreen

Tabel 14. Source Code Soal 1 HomeScreen.kt Modul 5

1.	package com.example.listcompose.ui.screen.home
2.	
3.	import android.content.Intent

4.	<code>import android.net.Uri</code>
5.	<code>import android.widget.Toast</code>
6.	<code>import androidx.compose.foundation.Image</code>
7.	<code>import androidx.compose.foundation.layout.Arrangement</code>
8.	<code>import androidx.compose.foundation.layout.Column</code>
9.	<code>import androidx.compose.foundation.layout.PaddingValues</code>
10.	<code>import androidx.compose.foundation.layout.Row</code>
11.	<code>import androidx.compose.foundation.layout.Spacer</code>
12.	<code>import androidx.compose.foundation.layout.fillMaxSize</code>
13.	<code>import androidx.compose.foundation.layout.fillMaxWidth</code>
14.	<code>import androidx.compose.foundation.layout.height</code>
15.	<code>import androidx.compose.foundation.layout.padding</code>
16.	<code>import androidx.compose.foundation.layout.size</code>
17.	<code>import androidx.compose.foundation.layout.width</code>
18.	<code>import androidx.compose.foundation.lazy.LazyColumn</code>
19.	<code>import androidx.compose.foundation.lazy.LazyRow</code>
20.	<code>import androidx.compose.foundation.lazy.items</code>
21.	<code>import</code> <code>androidx.compose.foundation.shape.RoundedCornerShape</code>
22.	<code>import androidx.compose.material3.Button</code>
23.	<code>import androidx.compose.material3.ButtonDefaults</code>
24.	<code>import androidx.compose.material3.Card</code>
25.	<code>import androidx.compose.material3.CardDefaults</code>

26.	<code>import androidx.compose.material3.MaterialTheme</code>
27.	<code>import com.example.listcompose.R</code>
28.	<code>import androidx.compose.material3.Scaffold</code>
29.	<code>import androidx.compose.material3.Text</code>
30.	<code>import androidx.compose.runtime.Composable</code>
31.	<code>import androidx.compose.runtime.LaunchedEffect</code>
32.	<code>import androidx.compose.runtime.getValue</code>
33.	<code>import androidx.compose.ui.Alignment</code>
34.	<code>import androidx.compose.ui.Modifier</code>
35.	<code>import androidx.compose.ui.draw.clip</code>
36.	<code>import androidx.compose.ui.graphics.Color</code>
37.	<code>import androidx.compose.ui.layout.ContentScale</code>
38.	<code>import androidx.compose.ui.platform.LocalContext</code>
39.	<code>import androidx.compose.ui.res.colorResource</code>
40.	<code>import androidx.compose.ui.res.painterResource</code>
41.	<code>import androidx.compose.ui.res.stringResource</code>
42.	<code>import androidx.compose.ui.text.font.FontWeight</code>
43.	<code>import androidx.compose.ui.text.style.TextOverflow</code>
44.	<code>import androidx.compose.ui.unit.dp</code>
45.	<code>import androidx.compose.ui.unit.sp</code>
46.	<code>import</code> <code> androidx.lifecycle.compose.collectAsStateWithLifecycle</code>
47.	<code>import androidx.lifecycle.viewmodel.compose.viewModel</code>

```
48. import androidx.navigation.NavController
49. import coil.compose.AsyncImage
50. import com.example.listcompose.domain.model.Film
51. import com.example.listcompose.ui.screen.HeaderScreen
52. import timber.log.Timber
53.
54. private const val TMDB_IMAGE_BASE_URL =
    "https://image.tmdb.org/t/p/w500"
55. @Composable
56. fun HomeScreen(
57.     navController: NavController,
58. ) {
59.     val context=LocalContext.current
60.     val factory= HomeViewModelFactory(
61.         context = context,
62.         pageInfo = stringResource(R.string.homepage)
63.     )
64.     val viewModel: HomeViewModel = viewModel(factory =
factory)
65.
66.     val films by
viewModel.films.collectAsStateWithLifecycle()
67.
68.     LaunchedEffect(films) {
```

```

69.         if (films.isEmpty()){
70.             Timber.d("Item Film yang Dikirimkan pada List:
              ${films.size}")
71.         }
72.     }
73.
74.     LaunchedEffect(Unit) {
75.         val homepageText =
context.getString(R.string.homepage)
76.         Toast.makeText(context, homepageText,
              Toast.LENGTH_SHORT).show()
77.     }
78.     Scaffold(
79.         topBar = {
80.             HeaderScreen(
81.                 title =
stringResource(R.string.app_title),
82.                 buttonText =
stringResource(R.string.languagebutton),
83.                 icon = R.drawable.language,
84.                 onClick = {
navController.navigate("language") }
85.             )
86.         }
87.     ) { innerPadding ->

```

```

88.         LazyColumn(
89.             modifier = Modifier
90.                 .fillMaxSize()
91.                 .padding(innerPadding)
92.         ) {
93.             item {
94.                 LazyRow(
95.                     contentPadding = PaddingValues(16.dp),
96.                     horizontalArrangement =
Arrangement.spacedBy(12.dp)
97.                 ) {
98.                     items(films) { film ->
99.                         HighlightCard(film)
100.                     }
101.                 }
102.             }
103.
104.             item {
105.                 Text(
106.                     text =
stringResource(R.string.app_about),
107.                     style =
MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),

```

```

108         modifier = Modifier.padding(start =
109         16.dp, bottom = 8.dp)
110     )
111 }
112
113 items(films) { film ->
114
115     val synopsisRes = when (film.id) {
116         274 -> R.string.synopsis_f01
117         812583 -> R.string.synopsis_f05
118         687163 -> R.string.synopsis_f03
119         1393326 -> R.string.synopsis_f02
120         1310741 -> R.string.synopsis_f04
121         else -> R.string.errormsg
122     }
123
124     val scoreRes = when (film.id) {
125         274 -> R.string.score_f01
126         812583 -> R.string.score_f05
127         687163 -> R.string.score_f03
128         1393326 -> R.string.score_f02
129         1310741 -> R.string.score_f04
130         else -> R.string.app_name

```

```

130     }
131
132     val imdbRes = when (film.id) {
133         274 -> R.string.imdb_f01
134         812583 -> R.string.imdb_f05
135         687163 -> R.string.imdb_f03
136         1393326 -> R.string.imdb_f02
137         1310741 -> R.string.imdb_f04
138         else -> R.string.app_name
139     }
140
141     MovieItem(
142         film = film,
143         synopsisRes = synopsisRes,
144         scoreRes = scoreRes,
145         onDetailClick = {
146             Timber.d("Tombol Detail dari Film
147 : ${film.title} Ditekan")
148
149         navController.navigate("detail/${film.id}")
150
151         },
152         onIMDBClick = {
153             Timber.d("Tombol IMDB dari Film :
154 ${film.title} Ditekan")

```



```

151         val intent =
            Intent(Intent.ACTION_VIEW,
                Uri.parse(context.getString(imdbRes)))
152         context.startActivity(intent)
153     }
154 )
155 }
156 }
157 }
158 }
159
160 @Composable
161 fun HighlightCard(film : Film) {
162     val bigImageRes = when(film.id){
163         274-> R.drawable.sotl
164         812583 -> R.drawable.wudm
165         687163 -> R.drawable.phm
166         1393326 -> R.drawable.gitc
167         1310741 -> R.drawable.sktp
168         else -> R.drawable.ic_launcher_background
169     }
170     Card(
171         modifier = Modifier

```

```

172         .width(280.dp)
173         .height(160.dp),
174         shape = RoundedCornerShape(16.dp)
175     ) {
176         Image(
177             painter = painterResource(id = bigImageRes),
178             contentDescription = null,
179             contentScale = ContentScale.Crop,
180             modifier = Modifier.fillMaxSize()
181         )
182     }
183 }
184
185 @Composable
186 fun MovieItem(
187     film: Film,
188     synopsisRes: Int,
189     scoreRes: Int,
190     onDetailClick: () -> Unit,
191     onIMDBClick: () -> Unit
192 ) {
193     Card(
194         modifier = Modifier

```

```

195         .fillMaxWidth()
196         .padding(horizontal = 16.dp, vertical = 8.dp),
197         colors =
CardDefaults.cardColors(colorResource(R.color.card)),
198         shape = RoundedCornerShape(16.dp)
199     ) {
200         Row(
201             modifier = Modifier
202                 .padding(12.dp)
203                 .fillMaxWidth()
204         ) {
205             AsyncImage(
206                 model =
"$TMDB_IMAGE_BASE_URL${film.posterPath}",
207                 contentDescription = "Poster
${film.title}",
208                 placeholder =
painterResource(R.drawable.ic_launcher_background),
209                 error =
painterResource(R.drawable.ic_launcher_background),
210                 modifier = Modifier
211                     .size(width = 110.dp, height = 160.dp)
212                     .clip(RoundedCornerShape(12.dp)),
213                 contentScale = ContentScale.Crop

```

```

214         )
215
216         Column(
217             modifier = Modifier
218                 .padding(start = 12.dp)
219                 .weight(1f)
220         ) {
221             Text(
222                 text = film.title,
223                 style =
224                 MaterialTheme.typography.titleMedium.copy(fontWeight =
225                 FontWeight.Bold),
226                 maxLines = 2,
227                 overflow = TextOverflow.Ellipsis
228             )
229
230             Spacer(modifier = Modifier.height(2.dp))
231
232             Text(
233                 text = "Release Date:
234                 ${film.releaseDate}",
235                 style =
236                 MaterialTheme.typography.bodySmall,

```

```

233         color =
MaterialTheme.colorScheme.outline
234     )
235
236     Text (
237         text = stringResource(synopsisRes),
238         style =
MaterialTheme.typography.bodySmall,
239         maxLines = 3,
240         overflow = TextOverflow.Ellipsis,
241         modifier = Modifier.padding(vertical =
4.dp)
242     )
243
244     Row(verticalAlignment =
Alignment.CenterVertically) {
245         Text (
246             text =
stringResource(R.string.rating),
247             style =
MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.Bold)
248         )
249         Spacer(modifier =
Modifier.width(4.dp))

```

```

250         Text(
251             text = stringResource(scoreRes),
252             style =
MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.Bold),
253             color = Color.DarkGray
254         )
255     }
256
257     Spacer(modifier = Modifier.height(8.dp))
258
259     Row(
260         modifier = Modifier.fillMaxWidth(),
261         horizontalArrangement =
Arrangement.spacedBy(8.dp)
262     ) {
263         Button(
264             onClick = onIMDBClick,
265             modifier = Modifier.weight(1f),
266             colors =
ButtonDefaults.buttonColors(colorResource(R.color.button))
,
267             contentPadding =
PaddingValues(0.dp)
268         ) {

```

```

269 Text(stringResource(R.string.imdb), fontSize = 11.sp,
    color = Color.Black)
270
271 Button(
272     onClick = onDetailClick,
273     modifier = Modifier.weight(1f),
274     colors =
ButtonDefaults.buttonColors(colorResource(R.color.button))
    ,
275     contentPadding =
PaddingValues(0.dp)
276 ) {
277 Text(stringResource(R.string.detail), fontSize = 11.sp,
    color = Color.Black)
278 }
279 }
280 }
281 }
282 }
283 }

```

HomeViewModel

Tabel 15. Source Code Soal 1 HomeViewModel.kt Modul 5

1.	package com.example.listcompose.ui.screen.home
2.	
3.	import androidx.lifecycle.ViewModel
4.	import androidx.lifecycle.viewModelScope
5.	import com.example.listcompose.data.repository.FilmRepository
6.	import com.example.listcompose.domain.model.Film
7.	import kotlinx.coroutines.flow.MutableStateFlow
8.	import kotlinx.coroutines.flow.SharingStarted
9.	import kotlinx.coroutines.flow.StateFlow
10.	import kotlinx.coroutines.flow.asStateFlow
11.	import kotlinx.coroutines.flow.stateIn
12.	import kotlinx.coroutines.launch
13.	import timber.log.Timber
14.	
15.	class HomeViewModel(16. private val repository: FilmRepository, 17. val pageInfo : String 18.) : ViewModel() { 19. val films: StateFlow<List<Film>> = repository.getAllMovies() 20. .stateIn(

21.	scope = viewModelScope,
22.	started =
	SharingStarted.WhileSubscribed(5000),
23.	initialValue = emptyList()
24.	)
25.	init {
26.	fetchMovieFromServer()
27.	}
28.	private fun fetchMovieFromServer(){
29.	viewModelScope.launch {
30.	val errorResult = repository.refreshMovies()
31.	if (errorResult != null) {
32.	Timber.e("SINKRONISASI GAGAL! Faktor Penyebab: \$errorResult")
33.	} else {
34.	Timber.d("SINKRONISASI SUKSES! Data film berhasil masuk ke Room.")
35.	}
36.	}
37.	}
38.	}

HomeViewModelFactory

Tabel 16. Source Code Soal 1 HomeViewModelFactory.kt Modul 5

1.	<code>package com.example.listcompose.ui.screen.home</code>
2.	
3.	<code>import android.content.Context</code>
4.	<code>import androidx.lifecycle.ViewModel</code>
5.	<code>import androidx.lifecycle.ViewModelProvider</code>
6.	<code>import</code> <code>com.example.listcompose.data.local.pref.SettingPreferences</code>
7.	<code>import</code> <code>com.example.listcompose.data.local.room.FilmDatabase</code>
8.	<code>import</code> <code>com.example.listcompose.data.remote.api.NetworkModule</code>
9.	<code>import</code> <code>com.example.listcompose.data.repository.FilmRepository</code>
10.	
11.	<code>class HomeViewModelFactory(</code>
12.	<code> private val context: Context,</code>
13.	<code> private val pageInfo : String</code>
14.	<code>): ViewModelProvider.Factory {</code>
15.	<code> override fun <T : ViewModel> create(modelClass:</code> <code>Class<T>): T {</code>
16.	<code> if(modelClass.isAssignableFrom(HomeViewModel::class.java))</code> <code>{</code>

17.	val apiService = NetworkModule.tmdbService
18.	val database = FilmDatabase.getDatabase(context)
19.	val filmDao = database.filmDao()
20.	val settingPreferences = SettingPreferences(context)
21.	val repository = FilmRepository(apiService, filmDao, settingPreferences)
22.	
23.	return HomeViewModel(repository, pageInfo) as T
24.	}
25.	throw IllegalArgumentException("Unknown ViewModel Class: \${modelClass.name}")
26.	}
27.	}

DetailScreen

Tabel 17. Source Code Soal 1 DetailScreen.kt Modul 5

1.	package com.example.listcompose.ui.screen.detail
2.	
3.	import android.widget.Toast
4.	import androidx.compose.foundation.Image
5.	import androidx.compose.foundation.layout.Box
6.	import androidx.compose.foundation.layout.Column

7.	<code>import androidx.compose.foundation.layout.Row</code>
8.	<code>import androidx.compose.foundation.layout.Spacer</code>
9.	<code>import androidx.compose.foundation.layout.fillMaxHeight</code>
10.	<code>import androidx.compose.foundation.layout.fillMaxSize</code>
11.	<code>import androidx.compose.foundation.layout.fillMaxWidth</code>
12.	<code>import androidx.compose.foundation.layout.height</code>
13.	<code>import androidx.compose.foundation.layout.padding</code>
14.	<code>import androidx.compose.foundation.layout.size</code>
15.	<code>import androidx.compose.foundation.layout.width</code>
16.	<code>import androidx.compose.foundation.rememberScrollState</code>
17.	<code>import</code> <code>androidx.compose.foundation.shape.RoundedCornerShape</code>
18.	<code>import androidx.compose.foundation.verticalScroll</code>
19.	<code>import androidx.compose.material3.Card</code>
20.	<code>import androidx.compose.material3.CardDefaults</code>
21.	<code>import</code> <code>androidx.compose.material3.CircularProgressIndicator</code>
22.	<code>import androidx.compose.material3.MaterialTheme</code>
23.	<code>import androidx.compose.material3.Scaffold</code>
24.	<code>import androidx.compose.material3.Text</code>
25.	<code>import androidx.compose.runtime.Composable</code>
26.	<code>import androidx.compose.runtime.LaunchedEffect</code>
27.	<code>import androidx.compose.runtime.getValue</code>

28.	import androidx.compose.runtime.remember
29.	import androidx.compose.ui.Modifier
30.	import androidx.compose.ui.draw.clip
31.	import androidx.compose.ui.layout.ContentScale
32.	import androidx.compose.ui.res.colorResource
33.	import androidx.compose.ui.res.painterResource
34.	import androidx.compose.ui.res.stringResource
35.	import androidx.compose.ui.text.font.FontWeight
36.	import androidx.compose.ui.unit.dp
37.	import androidx.compose.ui.unit.sp
38.	import com.example.listcompose.R
39.	import androidx.navigation.NavController
40.	import androidx.compose.ui.Alignment
41.	import androidx.compose.ui.platform.LocalContext
42.	import androidx.lifecycle.ViewModel
43.	import androidx.lifecycle.compose.collectAsStateWithLifecycle
44.	import androidx.lifecycle.viewmodel.compose.viewModel
45.	import com.example.listcompose.ui.screen.HeaderScreen
46.	import timber.log.Timber
47.	
48.	@Composable
49.	fun DetailScreen(

```

50.     filmId: Int,
51.     navController: NavController
52. ) {
53.     val context = LocalContext.current
54.
55.     val factory = DetailViewModelFactory(
56.         context = context,
57.         filmId = filmId,
58.         pageInfo = stringResource(R.string.detailpage)
59.     )
60.
61.     val viewModel: DetailViewModel = viewModel(factory =
factory)
62.     val film by
viewModel.film.collectAsStateWithLifecycle()
63.
64.     LaunchedEffect(film) {
65.         film?.let {
66.             Timber.d("Menampilkan Data Detail dari Film:
${it.title}")
67.         }
68.     }
69.
70.     LaunchedEffect(Unit) {

```

71.	Toast.makeText(context, viewModel.pageInfo,
	Toast.LENGTH_SHORT).show()
72.	}
73.	
74.	Scaffold(
75.	topBar = {
76.	HeaderScreen(
77.	title =
	stringResource(R.string.app_title),
78.	buttonText =
	stringResource(R.string.homebutton),
79.	icon = R.drawable.home,
80.	onButtonClick = {
	navController.popBackStack() }
81.	)
82.	}
83.	) { innerPadding ->
84.	val currentFilm = film
85.	
86.	if(currentFilm != null){
87.	val synopsisRes = when (currentFilm.id) {
88.	274 -> R.string.synopsis_f01
89.	812583 -> R.string.synopsis_f05
90.	687163 -> R.string.synopsis_f03

91.	1393326 -> R.string.synopsis_f02
92.	1310741 -> R.string.synopsis_f04
93.	else -> R.string.errormsg
94.	}
95.	
96.	val scoreRes = when (currentFilm.id) {
97.	274 -> R.string.score_f01
98.	812583 -> R.string.score_f05
99.	687163 -> R.string.score_f03
100.	1393326 -> R.string.score_f02
101.	1310741 -> R.string.score_f04
102.	else -> R.string.app_name
103.	}
104.	
105.	val reviewRes = when (currentFilm.id) {
106.	274 -> R.string.review_f01
107.	812583 -> R.string.review_f05
108.	687163 -> R.string.review_f03
109.	1393326 -> R.string.review_f02
110.	1310741 -> R.string.review_f04
111.	else -> R.string.errormsg
112.	}
113.	


```

114         val bigImageRes = when (currentFilm.id) {
115             274 -> R.drawable.sotl2
116             812583 -> R.drawable.wudm2
117             687163 -> R.drawable.phm2
118             1393326 -> R.drawable.gitc2
119             1310741 -> R.drawable.skt2
120             else -> R.drawable.ic_launcher_background
121         }
122
123         val imageRes = when (currentFilm.id) {
124             274 -> R.drawable.sotl
125             812583 -> R.drawable.wudm
126             687163 -> R.drawable.phm
127             1393326 -> R.drawable.gitc
128             1310741 -> R.drawable.skt
129             else -> R.drawable.ic_launcher_background
130         }
131
132         Column(
133             modifier = Modifier
134                 .fillMaxSize()
135                 .padding(innerPadding)
136                 .verticalScroll(rememberScrollState())

```

```

137         ) {
138             Image(
139                 painter = painterResource(id =
bigImageRes),
140                 contentDescription = null,
141                 modifier = Modifier
142                     .fillMaxWidth()
143                     .height(220.dp),
144                 contentScale = ContentScale.Crop
145             )
146
147             Column(modifier = Modifier.padding(16.dp))
148         {
149             Row(
150                 modifier =
Modifier.fillMaxWidth(),
151                 verticalAlignment =
Alignment.Bottom
152             ) {
153                 Column(modifier =
Modifier.weight(1f)) {
154                     Text(
155                         text = currentFilm.title,

```

```

156         style =
MaterialTheme.typography.headlineSmall.copy(fontWeight =
FontWeight.Bold)
157     )
158
159     Spacer(modifier =
Modifier.height(4.dp))
160
161     Text(
162         text = "Release Date:
${currentFilm.releaseDate}",
163         style =
MaterialTheme.typography.bodyMedium,
164         color =
MaterialTheme.colorScheme.outline
165     )
166
167     Spacer(modifier =
Modifier.height(12.dp))
168
169     Text(
170         text = stringResource(id =
synopsisRes),
171         style =
MaterialTheme.typography.bodyMedium,

```

```

172         lineHeight = 20.sp
173     )
174 }
175
176     Spacer(modifier =
177 Modifier.width(12.dp))
178
179     Image(
180         painter = painterResource(id =
181 imageRes),
182         contentDescription = null,
183         modifier = Modifier
184             .size(width = 120.dp,
185 height = 180.dp)
186             .clip(RoundedCornerShape(12.dp))
187             .fillMaxHeight(),
188         contentScale =
189 ContentScale.Crop
190     )
191 }
192
193     Spacer(modifier =
194 Modifier.height(16.dp))
195

```

```

191         Text(
192             text = "${stringResource(id =
R.string.rating)} ${stringResource(id = scoreRes)}",
193             style =
MaterialTheme.typography.titleLarge.copy(fontWeight =
FontWeight.Bold)
194         )
195
196         Spacer(modifier =
Modifier.height(16.dp))
197
198         Card(
199             modifier =
Modifier.fillMaxWidth(),
200             colors =
CardDefaults.cardColors(colorResource(R.color.card)),
201             shape = RoundedCornerShape(16.dp)
202         ) {
203             Column(modifier =
Modifier.padding(16.dp)) {
204                 Text(
205                     text = stringResource(id =
R.string.review),
206                     style =
MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold)

```

```

207         )
208         Spacer(modifier =
Modifier.height(4.dp))
209         Text(
210             text = stringResource(id =
reviewRes),
211             style =
MaterialTheme.typography.bodyMedium
212         )
213     }
214 }
215 }
216 }
217 } else {
218     Box(
219         modifier = Modifier
220             .fillMaxSize()
221             .padding(innerPadding),
222         contentAlignment = Alignment.Center
223     ) {
224         CircularProgressIndicator()
225     }
226 }
227 }

```

228.	}
------	---

DetailViewModel

Tabel 18. Source Code Soal 1 DetailViewModel.kt Modul 5

1.	package com.example.listcompose.ui.screen.detail
2.	
3.	import androidx.lifecycle.ViewModel
4.	import androidx.lifecycle.viewModelScope
5.	import com.example.listcompose.data.repository.FilmRepository
6.	import com.example.listcompose.domain.model.Film
7.	import kotlinx.coroutines.flow.MutableStateFlow
8.	import kotlinx.coroutines.flow.SharingStarted
9.	import kotlinx.coroutines.flow.StateFlow
10.	import kotlinx.coroutines.flow.asStateFlow
11.	import kotlinx.coroutines.flow.stateIn
12.	
13.	class DetailViewModel(14. private val repository: FilmRepository, 15. val filmId: Int, 16. val pageInfo: String 17.): ViewModel() { 18. val film: StateFlow<Film?> = repository.getMovieById(filmId)

19.	.stateIn(
20.	scope = viewModelScope,
21.	started =
	SharingStarted.WhileSubscribed(5000),
22.	initialValue = null
23.	)
24.	}

DetailViewModelFactory

Tabel 19. Source Code Soal 1 DetailViewModelFactory Modul 5

1.	package com.example.listcompose.ui.screen.detail
2.	
3.	import android.content.Context
4.	import androidx.lifecycle.ViewModel
5.	import androidx.lifecycle.ViewModelProvider
6.	import
	com.example.listcompose.data.local.pref.SettingPreferences
7.	import
	com.example.listcompose.data.local.room.FilmDatabase
8.	import
	com.example.listcompose.data.remote.api.NetworkModule
9.	import
	com.example.listcompose.data.repository.FilmRepository
10.	
11.	class DetailViewModelFactory(

12.	private val context : Context,
13.	private val filmId : Int,
14.	private val pageInfo : String
15.	): ViewModelProvider.Factory {
16.	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
17.	
	if(modelClass.isAssignableFrom(DetailViewModel::class.java
	)){
18.	val apiService = NetworkModule.tmdbService
19.	val database =
	FilmDatabase.getDatabase(context)
20.	val filmDao = database.filmDao()
21.	val settingPreferences =
	SettingPreferences(context)
22.	val repository = FilmRepository(apiService,
	filmDao, settingPreferences)
23.	
24.	return DetailViewModel(repository, filmId,
	pageInfo) as T
25.	}
26.	throw IllegalArgumentException("Unknown ViewModel
	Class: \${modelClass.name}")
27.	}
28.	}

LanguageScreen

Tabel 20. Source Code Soal 1 LanguageScreen.kt Modul 5

1.	package com.example.listcompose.ui.screen.language
2.	
3.	import android.widget.Toast
4.	import androidx.core.os.LocaleListCompat
5.	
6.	
7.	import androidx.appcompat.app.AppCompatActivity
8.	import androidx.compose.foundation.layout.*
9.	import androidx.compose.material3.*
10.	import androidx.compose.runtime.*
11.	import androidx.compose.ui.Alignment
12.	import androidx.compose.ui.Modifier
13.	import androidx.compose.ui.platform.LocalContext
14.	import androidx.compose.ui.res.stringResource
15.	import androidx.compose.ui.unit.dp
16.	import androidx.lifecycle.compose.collectAsStateWithLifecycle
17.	import androidx.lifecycle.viewmodel.compose.viewModel
18.	import androidx.navigation.NavController
19.	import com.example.listcompose.R
20.	import com.example.listcompose.ui.screen.HeaderScreen

21.	
22.	@Composable
23.	fun LanguageScreen(navController: NavController) {
24.	val context = LocalContext.current
25.	
26.	val factory = LanguageViewModelFactory(
27.	context = context,
28.	pageInfo = stringResource(R.string.languagepage)
29.	)
30.	val viewModel : LanguageViewModel = viewModel(factory
	= factory)
31.	
32.	val selectedOption by
	viewModel.selectedOption.collectAsStateWithLifecycle()
33.	
34.	LaunchedEffect(Unit) {
35.	Toast.makeText(context, viewModel.pageInfo,
	Toast.LENGTH_LONG).show()
36.	}
37.	
38.	Scaffold(
39.	topBar = {
40.	HeaderScreen(

41.	title =
	stringResource(R.string.app_title),
42.	buttonText =
	stringResource(R.string.homebutton),
43.	icon = R.drawable.home,
44.	onButtonClick = {
	navController.popBackStack() }
45.	)
46.	}
47.	) { innerPadding ->
48.	Column(
49.	modifier = Modifier
50.	.fillMaxSize()
51.	.padding(innerPadding)
52.	.padding(24.dp)
53.	) {
54.	LanguageOption(
55.	label = stringResource(R.string.english),
56.	selected = (selectedOption == "English"),
57.	onClick = {
	viewModel.selectLanguage("English") }
58.	)
59.	
60.	LanguageOption(

```

61.         label =
stringResource(R.string.indonesia),
62.         selected = (selectedOption == "Bahasa"),
63.         onClick = {
viewModel.selectLanguage("Bahasa") }
64.     )
65. }
66. }
67. }
68.
69. @Composable
70. fun LanguageOption(label: String, selected: Boolean,
onClick: () -> Unit) {
71.     Row(
72.         verticalAlignment = Alignment.CenterVertically,
73.         modifier = Modifier
74.             .fillMaxWidth()
75.             .padding(vertical = 4.dp)
76.     ) {
77.         RadioButton(
78.             selected = selected,
79.             onClick = onClick
80.         )
81.         Text(

```

82.	text = label,
83.	style = MaterialTheme.typography.bodyLarge,
84.	modifier = Modifier.padding(start = 8.dp)
85.	)
86.	}
87.	}

LanguageViewModel

Tabel 21. Source Code Soal 1 LanguageViewModel.kt Modul 5

1.	package com.example.listcompose.ui.screen.language
2.	
3.	import androidx.appcompat.app.AppCompatActivityDelegate
4.	import androidx.core.os.LocaleListCompat
5.	import androidx.lifecycle.ViewModel
6.	import androidx.lifecycle.viewModelScope
7.	import com.example.listcompose.data.local.pref.SettingPreferences
8.	import kotlinx.coroutines.flow.MutableStateFlow
9.	import kotlinx.coroutines.flow.StateFlow
10.	import kotlinx.coroutines.flow.asStateFlow
11.	import kotlinx.coroutines.flow.first
12.	import kotlinx.coroutines.launch
13.	
14.	class LanguageViewModel(

15.	private val settingPreferences: SettingPreferences,
16.	val pageInfo: String
17.	) : ViewModel() {
18.	private val _selectedOption =
	MutableStateFlow("English")
19.	val selectedOption: StateFlow<String> =
	_selectedOption.asStateFlow()
20.	
21.	init {
22.	viewModelScope.launch {
23.	val currentLangCode =
	settingPreferences.getLanguageSetting.first()
24.	_selectedOption.value = if (currentLangCode ==
	"in") "Bahasa" else "English"
25.	}
26.	}
27.	
28.	fun selectLanguage(language: String){
29.	_selectedOption.value = language
30.	val langCode = if (language == "Bahasa") "in" else
	"en"
31.	
32.	viewModelScope.launch {
33.	settingPreferences.saveLanguageSetting(langCode)

34.	val appLocale =
	LocaleListCompat.forLanguageTags (langCode)
35.	
	AppCompatActivity.setApplicationLocales (appLocale)
36.	}
37.	}
38.	}

LanguageViewModelFactory

Tabel 22. Source Code Soal 1 LanguageViewModelFactory Modul 5

1.	package com.example.listcompose.ui.screen.language
2.	
3.	import android.content.Context
4.	import androidx.lifecycle.ViewModel
5.	import androidx.lifecycle.ViewModelProvider
6.	import
	com.example.listcompose.data.local.pref.SettingPreferences
7.	
8.	class LanguageViewModelFactory(
9.	private val context: Context,
10.	private val pageInfo : String
11.	): ViewModelProvider.Factory {
12.	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {

13.	<code>if(modelClass.isAssignableFrom(LanguageViewModel::class.java)) {</code>
14.	<code> val settingPreferences =</code> <code>SettingPreferences(context)</code>
15.	<code> return LanguageViewModel(settingPreferences,</code> <code>pageInfo) as T</code>
16.	<code>}</code>
17.	<code> throw IllegalArgumentException("Unknown ViewModel</code> <code>Class: \${modelClass.name}")</code>
18.	<code>}</code>
19.	<code>}</code>

HeaderScreen

Tabel 23. Source Code Soal 1 HeaderScreen.kt Modul 5

1.	<code>package com.example.listcompose.ui.screen</code>
2.	
3.	<code>import androidx.annotation.DrawableRes</code>
4.	<code>import androidx.compose.foundation.clickable</code>
5.	<code>import androidx.compose.foundation.layout.Arrangement</code>
6.	<code>import androidx.compose.foundation.layout.Column</code>
7.	<code>import androidx.compose.foundation.layout.Row</code>
8.	<code>import androidx.compose.foundation.layout.fillMaxWidth</code>
9.	<code>import androidx.compose.foundation.layout.padding</code>
10.	<code>import androidx.compose.foundation.layout.size</code>

11.	import androidx.compose.material3.Icon
12.	import androidx.compose.material3.MaterialTheme
13.	import androidx.compose.material3.Surface
14.	import androidx.compose.material3.Text
15.	import androidx.compose.runtime.Composable
16.	import androidx.compose.ui.Alignment
17.	import androidx.compose.ui.Modifier
18.	import androidx.compose.ui.graphics.Color
19.	import androidx.compose.ui.res.colorResource
20.	import androidx.compose.ui.res.painterResource
21.	import androidx.compose.ui.res.stringResource
22.	import androidx.compose.ui.text.font.FontWeight
23.	import androidx.compose.ui.tooling.preview.Preview
24.	import androidx.compose.ui.unit.dp
25.	import com.example.listcompose.R
26.	import com.example.listcompose.ui.theme.ListComposeTheme
27.	
28.	@Composable
29.	fun HeaderScreen(
30.	title : String,
31.	buttonText : String,
32.	@DrawableRes icon : Int,
33.	onButtonClick : ()->Unit

```

34. ) {
35.     Surface(
36.         modifier = Modifier.fillMaxWidth(),
37.         color= colorResource(id = R.color.card),
38.         shadowElevation = 4.dp
39.     ) {
40.         Row(
41.             modifier = Modifier
42.                 .padding(horizontal = 24.dp, vertical =
20.dp)
43.                 .fillMaxWidth(),
44.             horizontalArrangement =
Arrangement.SpaceBetween,
45.             verticalAlignment = Alignment.CenterVertically
46.         ){
47.             Text(
48.                 text = title,
49.                 style =
MaterialTheme.typography.headlineMedium.copy(
50.                     fontWeight = FontWeight.Bold,
51.                     color = Color.Black
52.                 )
53.             )
54.             Column(

```

```

55.         horizontalAlignment =
Alignment.CenterHorizontally,
56.         modifier = Modifier.clickable {
onButtonClick() }
57.     ) {
58.         Icon(
59.             painter = painterResource(id = icon),
60.             contentDescription = null,
61.             modifier = Modifier.size(28.dp),
62.             tint = Color.Unspecified
63.         )
64.         Text(
65.             text = buttonText,
66.             style =
MaterialTheme.typography.labelMedium.copy(
67.                 fontWeight = FontWeight.Bold
68.             )
69.         )
70.     }
71. }
72. }
73. }
74.
75. @Preview(showBackground = true)

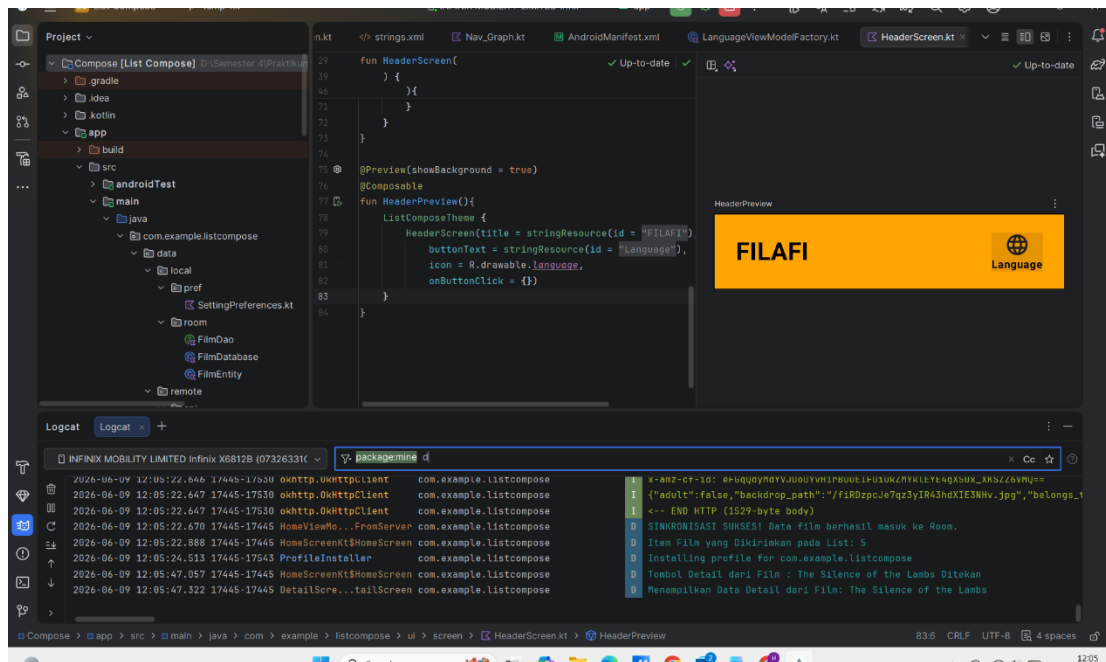
```

```

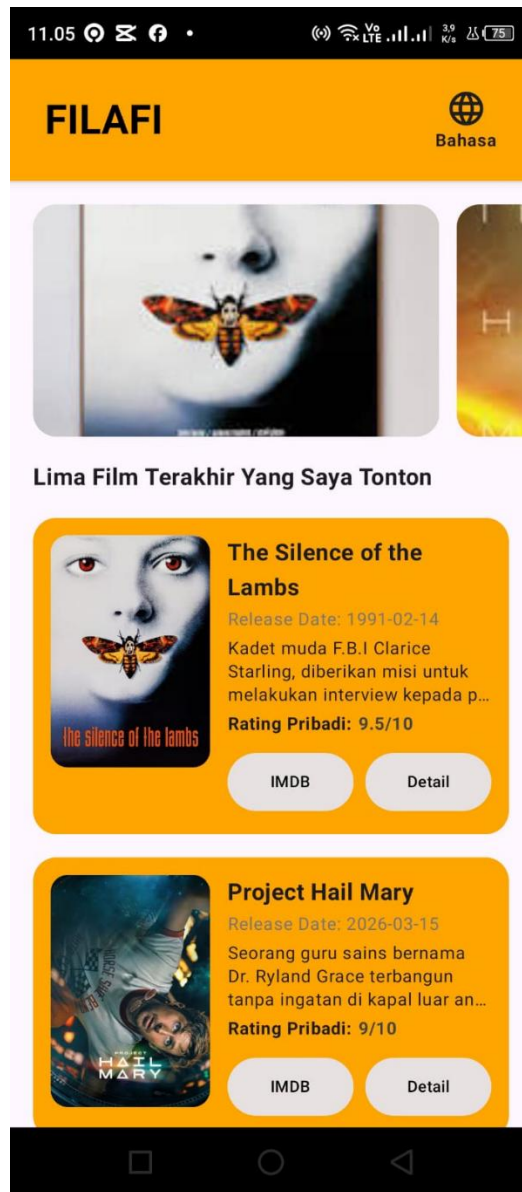
76. @Composable
77. fun HeaderPreview() {
78.     ListComposeTheme {
79.         HeaderScreen(title = stringResource(id =
R.string.app_title),
80.             buttonText = stringResource(id =
R.string.languagebutton),
81.             icon = R.drawable.language,
82.             onClick = {})
83.     }
84. }

```

B. Output Program



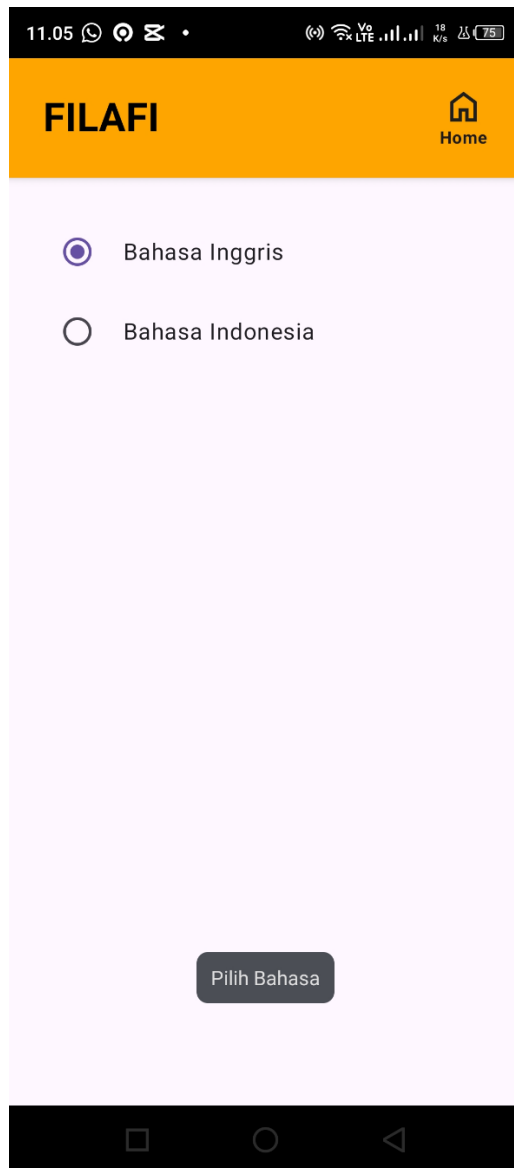
Gambar 1. Screenshot Hasil Logging Timber Soal 1 Modul 5



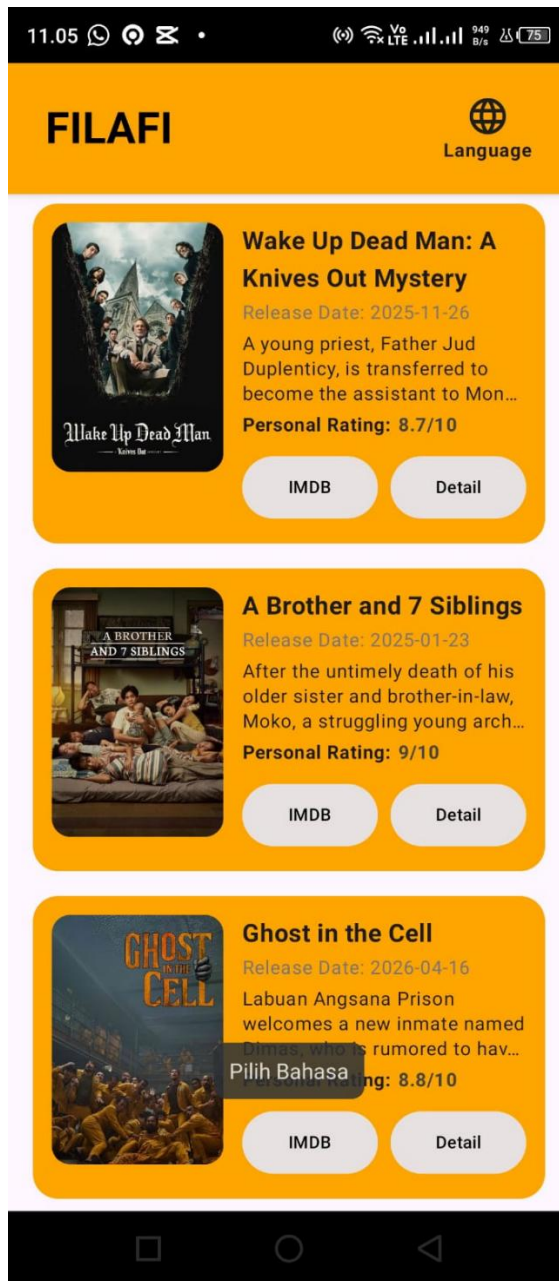
Gambar 2. Tampilan Halaman Home Aplikasi FILAFI Modul 5



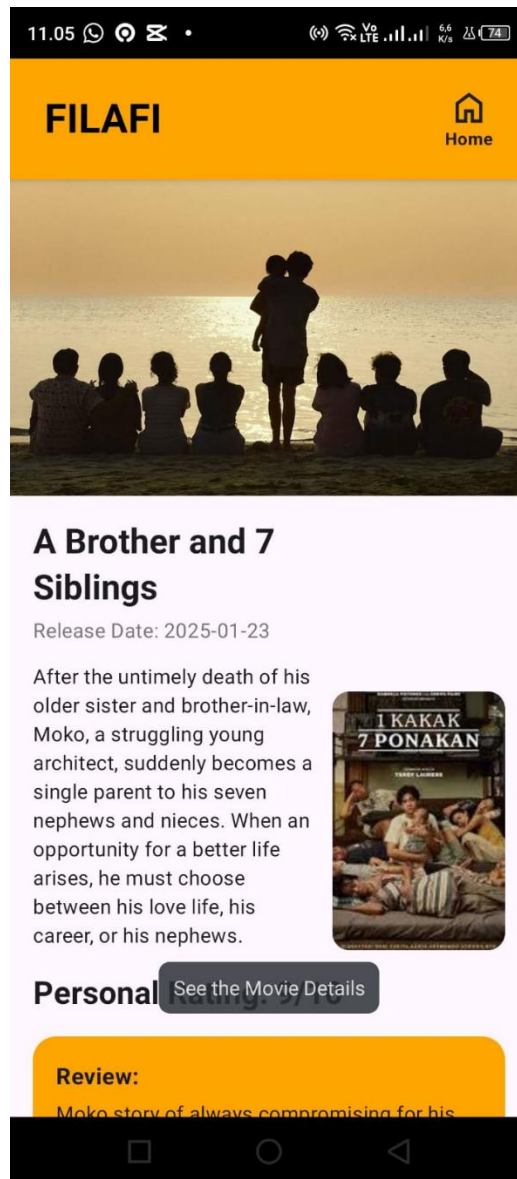
Gambar 3. Tampilan Halaman Detail Aplikasi FILAFI Modul 5



Gambar 4. Tampilan Halaman Pemilihan Bahasa Aplikasi FILAFI Modul 5



Gambar 5. Tampilan Halaman Home Berbahasa Inggris Aplikasi FILAFI Modul 5



Gambar 6. Tampilan Halaman Detail Berbahasa Inggris Modul 5

C. Pembahasan

Compose

MainActivity.kt

MainActivity berfungsi sebagai tempat dimulainya program lebih tepatnya pada onCreate yang akan menerima dan mengatur seluruh tampilan dari setContent berbasis fungsi composable. Di dalam setContent inilah tema aplikasi dari ListComposeTheme diterapkan, serta diinisialisasi variabel NavController untuk menangani proses navigasi

dan viewModel sebagai wadah penyedia data. Selain itu pada MainActivity juga diatur untuk menjadi host utama dari keseluruhan aplikasi dengan menempatkan komponen Nav_Graph di atas sebuah Surface yang mengatur warna latar dari aplikasi, di mana Nav_Graph tersebut akan menerima NavController dan pada modul ini ViewModel digantikan dengan StartDestination yang diatur menjadi home, agar nantinya dapat diteruskan dan digunakan pada seluruh proses navigasi halaman aplikasi.

MyApplication.kt

MyApplication memiliki peran sebagai kelas dasar yang menginisialisasi status global aplikasi sebelum komponen lain seperti *Activity* atau *Screen* dijalankan. Di dalamnya, diinisialisasi sebuah *companion object* yang menampung variabel statis `isDebug` dan `tmdbApiKey`, sehingga data tersebut dapat diakses dengan mudah dari mana saja di seluruh aplikasi. Proses inisialisasi dilakukan di dalam fungsi `onCreate()`, di mana variabel `isDebug` diberi nilai menggunakan `'and'` untuk memeriksa *flag* `FLAG_DEBUGGABLE` dari sistem, guna memastikan apakah aplikasi sedang berjalan dalam mode *development* atau *production*. Jika aplikasi terdeteksi dalam mode *debug*, maka library Timber akan ditanamkan menggunakan `DebugTree()`, yang bertujuan untuk mempermudah proses logging selama pengembangan tanpa khawatir log tersebut bocor saat aplikasi dirilis. Selain itu, variabel `tmdbApiKey` juga langsung diisi dengan mengambil nilai *String resource*, menjadikannya sebagai wadah terpusat untuk menyimpan kunci API TMDB yang nantinya siap digunakan oleh komponen data atau *repository* lainnya.

SettingPreference

SettingPreferences bertugas untuk mengelola penyimpanan data lokal yang ringan, khusus untuk menyimpan status pilihan bahasa aplikasi. Alur kerjanya memanfaatkan Jetpack DataStore untuk membaca dan menulis data. Kelas ini menyediakan variabel aliran data (Flow) yaitu `getLanguageSetting` untuk mengambil pengaturan bahasa secara otomatis dengan nilai bawaan bahasa Inggris jika belum ada pilihan serta menyediakan fungsi khusus yakni `saveLanguageSetting` untuk mengubah dan menyimpan kode bahasa baru saat pengguna mengganti bahasa aplikasi.

FilmEntity

FilmEntity memiliki peran untuk menjadi wadah struktur model data sekaligus merepresentasikan sebuah tabel bernama "movies" di dalam database lokal SQLite melalui library Room. Di dalam kelas ini, diinisialisasi beberapa variabel atribut film yang akan dipetakan menjadi kolom-kolom tabel menggunakan anotasi `@ColumnInfo`. Atribut id diatur menggunakan anotasi `@PrimaryKey` untuk menjadikannya sebagai identitas unik utama yang mencegah adanya duplikasi data. Atribut lainnya meliputi `title` untuk menyimpan judul film, serta `posterPath` dan `releaseDate` yang tipe datanya dibuat *nullable* (`String?`) guna mengantisipasi apabila data jalur gambar poster atau tanggal rilis belum tersedia dari API, sehingga aplikasi terhindar dari risiko *crash*.

FilmDao

FilmDao (Data Access Object) berperan sebagai jembatan untuk mengatur semua perintah manipulasi data pada tabel database. Alur kerjanya menyediakan fungsi reaktif menggunakan `<Flow>` untuk mengambil seluruh daftar film atau mencari film tertentu berdasarkan ID secara otomatis saat terjadi perubahan data. Kelas ini juga bertugas menyediakan fungsi untuk memasukkan data film baru (dengan sistem menimpa data lama jika sama) serta fungsi untuk menghapus seluruh isi database.

FilmDatabase

FilmDatabase berperan sebagai pondasi utama sekaligus gerbang transaksional yang menghubungkan aplikasi dengan database lokal SQLite berbasis Room. Kelas ini dibuat abstrak dengan mewarisi `RoomDatabase` dan mendaftarkan `FilmEntity` ke dalam susunan tabelnya pada versi pertama (`version = 1`). Di dalamnya, dideklarasikan fungsi abstrak `filmDao()` berisikan fungsi-fungsi query yang telah dibuat pada *interface* DAO.

Untuk memastikan pembuatan instansiasi database berjalan aman, diinisialisasi sebuah *companion object* yang menampung variabel `INSTANCE` dengan anotasi `@Volatile`. Atribut `@Volatile` ini krusial untuk memastikan bahwa nilai dari instansiasi database selalu sinkron dan dapat langsung terlihat oleh seluruh *thread* secara bersamaan. Pembuatan objek database dilakukan di dalam fungsi `getDatabase()`

menggunakan pola *Singleton* yang dibungkus oleh blok `synchronized(this)`. Logika ini memastikan bahwa pembuatan database dengan nama "filafi_db" hanya akan terjadi satu kali di awal siklus hidup aplikasi, sehingga mencegah pemborosan memori akibat pembuatan instansiasi database yang berulang-ulang dari berbagai komponen.

ApiResource.kt

ApiService berperan sebagai pendefinisian jalur komunikasi atau rute untuk mengambil data dari server internet. Tugasnya adalah menyediakan fungsi khusus untuk meminta detail data film tertentu dari API luar. Alur kerjanya berjalan dengan menerima masukan berupa ID film dan kode bahasa, lalu mengirimkan permintaan tersebut ke server untuk mengembalikan respon data film yang cocok.

NetworkModule.kt

NetworkModule merupakan sebuah komponen berstruktur *Singleton object* yang berperan sebagai pusat penyedia konfigurasi jaringan internet untuk aplikasi. Di awal komponen, diinisialisasi konstanta `BASE_URL` yang menyimpan alamat dasar dari API TMDb. Untuk mendukung proses pengembangan, diinisialisasi variabel `loggingInterceptor` yang memanfaatkan `HttpLoggingInterceptor`, di mana level pencatatannya diatur secara dinamis menggunakan variabel `MyApplication.isDebug` agar hanya menampilkan detail body pesan data saat mode *debug* dan dimatikan saat aplikasi dirilis.

Selain itu, diinisialisasi variabel `authInterceptor` yang bertugas menyisipkan parameter `api_key` secara otomatis ke dalam baris URL permintaan menggunakan nilai kunci yang diambil dari `MyApplication.tmdbApiKey`. Kedua interceptor ini, bersama dengan pengaturan batas waktu koneksi atau `timeout` selama 30 detik, kemudian dimasukkan ke dalam konfigurasi `okHttpClient`. Proses serialisasi data JSON dari server ditangani oleh variabel `json` yang diatur dengan parameter `ignoreUnknownKeys = true` guna mencegah kegagalan aplikasi apabila API mengirimkan atribut baru yang belum terdaftar pada model data. Terakhir, dilakukan inisialisasi variabel `retrofit` untuk merakit seluruh konfigurasi dasar ke dalam komponen Retrofit, hingga menghasilkan

variabel layanan `tmdbService` yang siap dipanggil dari mana saja untuk memproses fungsi-fungsi dari `ApiService`.

FilmDto

`FilmDto` (Data Transfer Object) berperan sebagai wadah model data sementara yang bertugas untuk menerima dan membaca struktur data mentah berbentuk JSON yang dikirimkan oleh server internet (API TMDB). Karakteristik utamanya menggunakan anotasi `@Serializable` agar data JSON bisa diurai otomatis, serta `@SerializedName` untuk mencocokkan nama kunci (*key*) dari server ke dalam variabel aplikasi. `FilmDTO` direturn oleh `ApiService` sehingga ketika dilakukan pengambilan data dari API, maka datanya akan dirubah ke objek pada `FilmDto`.

ErrorStatus

`ErrorStatus` adalah class ENUM yang nantinya digunakan untuk mendeteksi jenis error yang terjadi pada repository, yang mana jika errornya `NO_NETWORK` berarti tidak ada jaringan internet, jika `UNKNOWN_ERROR` itu error yang tidak diduga, sementara misalnya `SERVER_ERROR` itu menandakan jika terjadi masalah pada pengambilan data.

Film

`Film.kt` berperan sebagai layer model dari aplikasi FILAFI, di mana file ini difungsikan untuk merepresentasikan data film untuk diteruskan menuju ke layer UI. Di file ini juga terdapat function `FilmEntity.toDomain()` untuk menerima nilai dari `FilmEntity` yang kemudian diubah ke objek `Film` sebelum dikirimkan ke halaman UI.

Nav_Graph

`Nav_Graph.kt` berfungsi sebagai pusat kendali utama untuk mengatur seluruh alur navigasi dan perpindahan antar-halaman di dalam aplikasi. Tugasnya adalah mendefinisikan rute-rute (routes) halaman yang tersedia dan mengelola argumen yang dikirim saat berpindah layar, di mana alur kerjanya memanfaatkan komponen `NavHost` yang menetapkan rute "home" sebagai halaman pertama yang dimuat, rute "detail/{filmId}" yang diatur secara spesifik untuk menangkap argumen ID film bertipe

data Integer dari jalur navigasi lalu meneruskannya ke DetailScreen, serta rute "language" untuk memanggil LanguageScreen. File ini berkaitan erat dengan seluruh komponen Screen UI sebagai perakitan halaman yang memberikan objek NavController agar tombol navigasi di setiap layar berfungsi, serta menjadi lokasi strategis yang berinteraksi dengan HomeViewModelFactory dan DetailViewModelFactory untuk menyisipkan parameter parameter sebelum masuk ke ViewModel agar kelas UI menjadi lebih bersih, hingga akhirnya file ini dipanggil di dalam gerbang utama aplikasi (MainActivity) agar seluruh sistem navigasi Jetpack Compose dapat aktif berjalan sejak aplikasi pertama kali dimuat.

HomeScreen

HomeScreen.kt berperan untuk mengatur susunan tampilan utama dari halaman beranda aplikasi film. Tugas utamanya adalah merakit komponen tata letak menggunakan struktur Scaffold yang memuat HeaderScreen di bagian atas, serta menggunakan LazyColumn dan LazyRow agar daftar film dapat digulir dengan lancar dan hemat memori. Alur kerjanya dimulai dengan menginisialisasi HomeViewModel melalui HomeViewModelFactory untuk menyuntikkan data teks toast berbasis String resource dari Strings.xml.

Selanjutnya, data daftar film diambil dengan metode yang lebih hemat daya karena pengambilan data tidak akan dilakukan ketika bagian layar sedang tidak dilihat pengguna menggunakan collectAsStateWithLifecycle(), di mana jumlah data yang mengalir akan dipantau melalui komponen pencatatan LaunchedEffect dengan library Timber. Di dalam tata letak, fungsi komponen HighlightCard dijalankan secara horizontal untuk menampilkan gambar berukuran besar berdasarkan pencocokan ID film. Sementara itu, komponen MovieItem disusun secara vertikal ke bawah untuk menampilkan detail informasi setiap film, di mana gambar poster dimuat langsung dari server internet menggunakan komponen AsyncImage (Coil), data teks sinopsis, skor, serta tautan IMDB disuntikkan secara dinamis melalui pengecekan kondisi ID film, serta disediakan logika klik tombol untuk memicu navigasi perpindahan halaman maupun menjalankan Intent guna membuka browser luar.

HomeViewModel

HomeViewModel.kt berperan sebagai jembatan yang mengatur perputaran data antara bagian Repository dan tampilan HomeScreen, dengan tugas menyediakan aliran data film reaktif serta memicu proses sinkronisasi data dari internet saat halaman pertama kali dibuka. Alur kerjanya dimulai ketika kelas ini diinisialisasi untuk menerima suntikan parameter FilmRepository dan teks pageInfo, di mana variabel data films dideklarasikan menggunakan StateFlow<List<Film>> yang mengambil data langsung dari fungsi repository.getAllMovies(). Aliran data ini dikonversi menggunakan metode stateIn di dalam ruang lingkup viewModelScope dengan strategi WhileSubscribed(5000) agar pengiriman data otomatis berhenti jika layar tidak dilihat pengguna selama lebih dari 5 detik demi menghemat daya baterai. Bersamaan dengan itu, blok fungsi init langsung memicu fungsi fetchMovieFromServer() untuk memerintahkan Repository mengunduh data film terbaru dari server internet, lalu mencatat status hasilnya ke dalam log sukses atau gagal via library Timber.

HomeViewModelFactory

HomeViewModelFactory.kt berperan sebagai factory class yang bertugas untuk merakit dan menyuntikkan berbagai dependensi data yang diperlukan oleh HomeViewModel sebelum komponen tersebut digunakan di halaman beranda. Alur kerjanya berjalan di dalam fungsi create(), di mana sistem pertama-tama melakukan pengecekan kondisi untuk memastikan bahwa kelas ViewModel yang diminta memang benar merupakan tipe dari HomeViewModel. Jika kondisi terpenuhi, fungsi ini akan menginisialisasi komponen jaringan apiService dari NetworkModule, membuat atau mengambil instansiasi database lokal melalui FilmDatabase.getDatabase(context) untuk mendapatkan objek filmDao, serta menyiapkan manajemen penyimpanan lokal menggunakan SettingPreferences(context). Semua komponen data tersebut kemudian disatukan ke dalam baris pembuatan objek FilmRepository, sebelum akhirnya pabrikasi ini mengembalikan instansiasi utuh HomeViewModel dengan membawa suntikan objek repository serta parameter teks pesan pageInfo yang siap dipakai oleh lapisan HomeScreen.

DetailScreen

DetailScreen.kt berperan untuk menyusun komponen antarmuka yang menampilkan informasi lengkap dan detail dari sebuah film yang dipilih pengguna. Tugas utamanya adalah merakit tata letak halaman menggunakan Scaffold yang mengunci komponen HeaderScreen di bagian atas beserta tombol kembali berbasis `popBackStack()`, serta mengemas seluruh konten informasi ke dalam Column yang dibuat agar bisa digulir menggunakan fungsi `verticalScroll()`. Alur kerjanya dimulai dengan menginisialisasi `DetailViewModel` lewat `DetailViewModelFactory` untuk menyuntikkan data `filmId` dan teks `String resource` dari halaman detail guna memunculkan pesan toast. Selanjutnya, data objek film spesifik diamati dari `ViewModel` menggunakan delegasi `collectAsStateWithLifecycle()` dan dipantau log perubahannya melalui library `Timber` di dalam `LaunchedEffect`. Di dalam struktur tampilannya, sistem melakukan pengecekan kondisi; jika data film belum termuat maka layar akan memunculkan indikator pemuatan data berupa `CircularProgressIndicator()`, namun jika data sudah tersedia, layar akan langsung merender komponen gambar poster besar, judul, tanggal rilis, teks sinopsis, gambar poster lokal kecil, teks rating skor, hingga komponen `Card` yang menyuntikkan teks ulasan atau review untuk film secara dinamis berdasarkan pencocokan ID film yang dikirimkan.

DetailViewModel

`DetailViewModel.kt` berperan sebagai penyedia data spesifik untuk halaman detail atau `DetailScreen` dengan tugas mengamati dan menyalurkan data satu objek film berdasarkan ID yang dipilih pengguna. Alur kerjanya dimulai saat kelas ini diinisialisasi melalui `Factory` untuk menerima suntikan parameter berupa `FilmRepository`, data dinamis `filmId`, dan teks informasi `pageInfo`, di mana variabel data film dideklarasikan menggunakan `StateFlow<Film?>` dengan nilai awal kosong (`null`). Aliran data ini mengambil data film secara reaktif dari fungsi `repository.getMovieById(filmId)` dan dikonversi menggunakan metode `stateIn` di dalam ruang lingkup `viewModelScope` dengan strategi `WhileSubscribed(5000)`, yang

bertugas menghentikan pemantauan data secara otomatis jika pengguna meninggalkan halaman detail selama lebih dari 5 detik demi menghemat konsumsi daya baterai handphone.

DetailViewModelFactory

DetailViewModelFactory.kt berfungsi sebagai factory class yang bertugas untuk merakit dan menyuntikkan seluruh dependensi data yang diperlukan oleh DetailViewModel sebelum masuk ke lapisan antarmuka. Alur kerjanya berjalan di dalam fungsi create(), di mana sistem akan melakukan pengecekan tipe kelas untuk memastikan kesesuaian dengan DetailViewModel. Jika cocok, fungsi ini akan menyiapkan komponen jaringan apiService dari NetworkModule, membangun akses database lokal untuk mengambil objek filmDao, serta menyiapkan SettingPreferences(context) sebelum seluruh komponen data tersebut disatukan ke dalam objek FilmRepository, hingga akhirnya mengembalikan objek instansiasi utuh DetailViewModel lengkap dengan bawaan data filmId serta pageInfo.

LanguageScreen

LanguageScreen.kt berperan untuk menyusun struktur tampilan halaman pengaturan bahasa aplikasi. Tugas utamanya adalah merakit kerangka halaman menggunakan Scaffold yang memuat komponen HeaderScreen di bagian atas lengkap dengan tombol kembali berbasis popBackStack(), serta menampilkan daftar pilihan bahasa menggunakan Column(). Alur kerjanya dimulai dengan menginisialisasi LanguageViewModel melalui LanguageViewModelFactory untuk menyuntikkan teks String resource dari halaman pengaturan bahasa guna menampilkan pesan toast di dalam LaunchedEffect. Selanjutnya, status opsi bahasa yang sedang aktif diambil secara reaktif menggunakan collectAsStateWithLifecycle(). Di dalam komponen utamanya, halaman ini memanggil fungsi komponen LanguageOption sebanyak dua kali untuk merender Row() berisi komponen pilihan radio button dan teks label bahasa, di mana masing-masing opsi disuntikkan logika onClick yang memicu fungsi

`viewModel.selectLanguage()` dengan mengirimkan data string "English" atau "Bahasa" guna memperbarui kondisi pilihan bahasa aplikasi.

LanguageViewModel

`LanguageViewModel.kt` berperan sebagai pengatur logika dan state pada halaman pengaturan bahasa aplikasi. Tugas utamanya adalah membaca preferensi bahasa yang tersimpan di memori lokal serta menerapkan perubahan bahasa baru ke seluruh sistem aplikasi. Alur kerjanya dimulai ketika kelas ini diinisialisasi untuk menerima suntikan parameter berupa `SettingPreferences` dan teks informasi halaman `pageInfo`. Di dalam blok fungsi `init`, `viewModelScope.launch` langsung dijalankan untuk mengambil data pengaturan bahasa yang tersimpan di `DataStore` menggunakan fungsi `first()`, lalu mengubah nilai variabel privat `_selectedOption` menjadi "Bahasa" jika kode bahasa terdeteksi "in", atau "English" untuk kondisi selain itu. Selanjutnya, saat pengguna memilih opsi bahasa baru di layar, fungsi `selectLanguage()` akan dipicu untuk langsung memperbarui nilai `_selectedOption` dan menentukan kode bahasa yaitu `langCode` yang sesuai. Alur berikutnya berjalan di dalam coroutine untuk menyimpan kode bahasa baru tersebut kembali ke dalam lokal `DataStore` via `settingPreferences.saveLanguageSetting()`, sekaligus memperbarui konfigurasi bahasa sistem aplikasi secara menyeluruh menggunakan komponen `LocaleListCompat` dan perintah `AppCompatDelegate.setApplicationLocales()`.

LanguageViewModelFactory

`LanguageViewModelFactory.kt` berfungsi sebagai factory class yang bertugas untuk merakit dependensi yang dibutuhkan oleh `LanguageViewModel` sebelum dijalankan di halaman pengaturan bahasa. Alur kerjanya berjalan di dalam fungsi `create()`, di mana sistem akan melakukan pengecekan tipe kelas untuk memastikan bahwa `ViewModel` yang diminta adalah benar tipe dari `LanguageViewModel`. Jika kondisi tersebut terpenuhi, fungsi ini akan langsung menginstansiasi objek `SettingPreferences(context)` sebagai komponen manajemen penyimpanan lokal yang dibutuhkan untuk mengatur preferensi bahasa aplikasi. Terakhir, fungsi ini mengembalikan objek instansiasi utuh `LanguageViewModel` lengkap dengan

membawa suntikan objek preferensi lokal serta parameter teks informasi pesan pageInfo yang siap digunakan oleh lapisan antarmuka.

HeaderScreen

HeaderScreen berfungsi untuk mengatur tampilan header statis yang nantinya dapat dipanggil pada halaman home, language, dan detail, di mana function HeaderScreen diatur agar menerima parameter berupa judul, teks untuk tombol, icon yang diatur agar menerima resource drawable, serta logika jika tombol dilakukan klik. Pengaturan tampilan diatur menggunakan Surface yang diberi pengaturan warna dan dibuat memaksimalkan lebar yang tersedia pada layar. Dengan komponennya disusun menggunakan Row, Text, dan Icon yang dibuat agar bisa diklik.

Untuk metode catching strategy yang digunakan adalah Single Source Of Truth atau catching strategy dengan konsep satu sumber data secara mutlak. Implementasinya pada FILAFI sendiri adalah dengan menjadikan Room sebagai sumber data tunggal sehingga data dari internet yang diunduh langsung diteruskan oleh Retrofit menuju ke Room, yang keunggulannya adalah data dari TMDB bisa tetap terlihat pada aplikasi yang pada kasus FILAFI adalah data poster film, tanggal rilis, dan judul dari film, meskipun saat membuka aplikasi FILAFI perangkat Android sedang offline. Karena data-data dari TMDB tersebut sangat krusial bagi aplikasi list film seperti FILAFI, sehingga itulah mengapa metode catching inilah yang saya pilih.

TAUTAN GIT

<https://github.com/HamkaArifani/Praktikum-Pemrograman-Mobile>